정규 표현식

정규표현식

- 정규 표현식(Regular Expression)
  - 문자열 내에서 패턴을 쉽게 찾는 방법
  - 복잡한 문자열 조건을 짧게 표현할 수 있음
  - 이메일, 전화번호처럼 형식이 있는 텍스트에 강함
  - 일반적으로 '정규식' 이라고 줄여서 부름

- 정규 표현식 필요성

```python
data = """
park 800905-1049118
kim  700905-1059119
"""

result = []
for line in data.split("\n"):
    word_result = []
    for word in line.split(" "):
        if len(word) == 14 and word[:6].isdigit() and word[7:].isdigit():
            word = word[:6] + "-" + "*******"
        word_result.append(word)
    result.append(" ".join(word_result))
print("\n".join(result))
```

```python
import re

data = """
park 800905-1049118
kim  700905-1059119
"""

pat = re.compile("(\d{6})[-]\d{7}")
print(pat.sub("\g<1>-*******", data))
```

re 모듈

- 파이썬은 정규 표현식을 지원하기 위해 re(regular expression의 약어) 모듈 제공
  - 파이썬을 설치할 때 자동으로 설치되는 기본 라이브러리

```
# 모듈 불러오기
import re

# 컴파일
pat = re.compile('abc*')
```

- re.compile 함수로 컴파일 해 얻은 패턴 객체를 사용해 이후 작업 수행
- 패턴 객체로 문자열을 검색하기 위해 4가지 메서드를 제공함.

메소드	기능
match	문자열 맨 앞부터 정규식과 매치되는지 확인 (매치되지 않으면 None)
search	문자열 전체를 대상으로 정규식과 매치되는지 확인 (매치되지 않으면 None)
findall	정규식과 매치되는 모든 문자열을 리스트 형태로 반환
finditer	정규식과 매치되는 모든 문자열을 반복 가능한 객체로 반환

핵심 문법 살펴보기

패턴	설명
.	임의의 한 문자 (줄바꿈 문자 제외)
^	문자열의 시작
$	문자열의 끝
*	0회 이상 반복
+	1회 이상 반복
?	0회 또는 1회
{n}	n회 반복
{n,}	n회 이상 반복
{n,m}	n회 이상 m회 이하 반복
[]	문자 집합, 괄호 안의 문자 중 하나와 매치
|	OR 연산자
₩	이스케이프 문자
()	그룹을 지정
₩d	숫자와 매치, [0-9]
₩D	숫자가 아닌 것과 매치, [^0-9]
₩s	공백 문자와 매치, [₩t₩n₩r₩f₩v]
₩S	공백 문자가 아닌 것과 매치, [^ ₩t₩n₩r₩f₩v]
₩w	문자 + 숫자와 매치, [a-zA-Z0-9_]
₩W	문자 + 숫자가 아닌 것과 매치, [^a-zA-Z0-9_]

match vs search

- match 메서드는 문자열 맨 앞부터 정규식과 매치되는지 확인
- 매치되지 않으면 None 값을 반환
  - 예) 맨 앞에 'a' 문자가 있는지 확인

```
string = 'I have a Dream.'
pat = re.compile('a')
print(pat.match(string))   # None
```

- search 메서드는 문자열 전체를 대상으로 정규식과 매치되는지 확인
  - 예) 문자열에 'a' 문자가 있는지 확인

```
string = 'I have a Dream.'
pat = re.compile('a')
print(pat.search(string))  # <re.Match object; span=(3, 4), match='a'>
```

매치된 결과 확인

- 매치된 결과를 다음과 같은 메소드로 확인

메소드	기능
group	매치된 문자열을 반환
start	매치된 문자열의 시작 위치를 반환
end	매치된 문자열의 끝 위치를 반환
span	매치된 문자열의 (시작, 끝)에 해당하는 튜플 형태를 반환

```
string = 'I have a Dream.'
pat = re.compile('Dream')
result = pat.search(string)
print(result.group())        # Dream
print(result.start())        # 9
print(result.end())          # 14
print(result.span())         # (9, 14)
```

findall vs finditer

- findall 메서드는 정규식과 매치되는 모든 문자열을 리스트 형태로 반환

```
string = 'I have a Dream.'
pat = re.compile('a')
print(pat.findall(string))  # ['a', 'a', 'a']
```

```
string = 'I have a Dream.'
pat = re.compile('ave')
print(pat.findall(string))  # ['ave']
```

- finditer 메서드는 정규식과 매치되는 모든 문자열을 반복 가능한 객체로 반환

```
string = 'I have a Dream.'
pat = re.compile('a')
result = pat.finditer(string)
for i in result:
    print(i)
```

```
<re.Match object; span=(3, 4), match='a'>
<re.Match object; span=(7, 8), match='a'>
<re.Match object; span=(12, 13), match='a'>
```

정규 표현식 함수

함수	설명	사용 예
match()	문자열의 처음부터 정규식과 매치되는지 확인	import re pattern = re.compile(r'₩d+') match = pattern.match('123abc') if match: print(match.group()) # 123
search()	문자열 전체를 검색하여 정규식과 매치되는지 확인	pattern = re.compile(r'₩d+') match = pattern.search('abc123') if match: print(match.group()) # 123
findall()	정규식과 매치되는 모든 부분을 리스트로 반환	pattern = re.compile(r'₩d+') matches = pattern.findall('abc123def456') print(matches) # ['123', '456']
sub()	정규식과 매치되는 부분을 다른 문자열로 대체	pattern = re.compile(r'₩d+') result = pattern.sub('NUMBER', 'abc123def456') print(result) # abcNUMBERdefNUMBER

result = pattern.sub('NUMBER', 'abc123def456') ➡ result = re.sub(pattern, 'NUMBER', 'abc123def456')

같은 결과를 얻는 코드

패턴의 몇 가지 예

패턴	설명	예시
₩d+	숫자가 하나 이상인 부분과 매치	'123abc'.match(r'₩d+') -> '123'
₩D+	숫자가 아닌 부분과 매치	'abc123'.match(r'₩D+') -> 'abc'
₩s+	공백 문자가 하나 이상인 부분과 매치	'abc 123'.match(r'₩s+') -> ' '
₩S+	공백 문자가 아닌 부분과 매치	'abc123'.match(r'₩S+') -> 'abc123'
₩w+	문자와 숫자가 하나 이상인 부분과 매치	'abc123'.match(r'₩w+') -> 'abc123'
₩W+	문자와 숫자가 아닌 부분과 매치	'abc123'.match(r'₩W+') -> None
[a-z]	소문자 a부터 z까지 중 하나와 매치	'abc'.match(r'[a-z]') -> 'a'
[^a-z]	소문자 a부터 z까지 아닌 것과 매치	'ABC'.match(r'[^a-z]') -> 'A'
^abc	문자열의 시작이 'abc'와 매치	'abc123'.match(r'^abc') -> 'abc'
abc$	문자열의 끝이 'abc'와 매치	'123abc'.match(r'abc$') -> 'abc'
a|b	'a' 또는 'b'와 매치	'a'.match(r'a|b') -> 'a'
(abc)	'abc'와 매치되는 그룹을 생성	'abc'.match(r'(abc)') -> 'abc'
₩bword₩b	단어 경계에 있는 'word'와 매치	'word wordy'.match(r'₩bword₩b') -> 'word'

정규 표현식 사용하기

웹에서 수집한 텍스트 데이터에서 원하는 패턴의 텍스트를 찾거나 다른 텍스트로 대체할 수 있음

```
import re

string = 'hello hjk790@gmail.com 010-222-3333 world 02-5678-9988'

phone_pattern = re.compile(r'₩d{2,3}-₩d{3,4}-₩d{4}') # raw, 위의 예에서 전화번호를 찾는 패턴
phone_list = phone_pattern.findall(string)  # re.findall(phone_pattern, string)와 같음
print(phone_list)

email_pattern = re.compile(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+₩.[a-zA-Z]{2,}') # 이메일 패턴
email_list = re.findall(email_pattern, string)
print(email_list)
```